**Programming in Java**

**Unit-I:** Introduction to Java-Features of java-Object Oriented Concepts-Lexical Issues- Data Types- Variables-Arrays-Operators-Control Statements.

**Introduction to Java**

- **JAVA** was developed by James Gosling at **Sun Microsystems** Inc in the May **1995** and later acquired by Oracle Corporation.
- Currently, Java is used in mobile devices, internet programming, games, e-business, etc.

**Java Terminology:**

Before learning Java, one must be familiar with these common terms of Java.

**1. Java Virtual Machine(JVM):** This is generally referred to as JVM. There are three execution phases of a program. They are written, compile and run the program.

- Writing a program is done by a java programmer.
- The compilation is done by the **JAVAC** compiler which is a primary Java compiler included in the Java development kit (JDK). It takes the Java program as input and generates bytecode as output.
- In the Running phase of a program, **JVM** executes the bytecode generated by the compiler.

**2. Bytecode in** the **Development Process:**

As discussed, the Javac compiler of JDK compiles the java source code into bytecode so that it can be executed by JVM. It is saved as **.class** file by the compiler.

**3. Java development kit** that includes everything including compiler, Java Runtime Environment (JRE), java debuggers, java docs, etc. For the program to execute in java, we need to install JDK on our computer in order to create, compile and run the java program

**4. JRE** includes a browser, JVM, applet support, and plugins. For running the java program, a computer needs JRE.

**5. Garbage Collector:** In Java, programmers can't delete the objects. To delete or recollect that memory JVM has a program called Garbage Collector. Garbage Collectors can recollect the objects that are not referenced.

**Main Features of Java**

- The programs written in Java language, after compilation, are converted into an intermediate level language called the **bytecode**.
- This makes java highly portable as its bytecodes can be run on any machine by an interpreter called the **Java Virtual Machine (JVM).**
- Every Operating System has a different JVM but the output they produce after the execution of bytecode is the same across all the operating systems. This is why Java is known as **a platform-independent** language.
- Java is a **simple language**

- Java is easy to learn and its syntax is **easy to understand**.
- It is based on C++.
- Java has removed many confusing and rarely-used features
  e.g. explicit pointers, operator overloading etc.
- Java provides an automatic garbage collector.
- **Java is an object-oriented programming language**
  - OOP makes the complete program simpler by dividing it into a number of objects.
  - The objects can communicate from one function to another.
  - We can easily modify data and function's as per the requirements of the program.
  - Basic Concepts of OOPs are: object, class, Inheritance, Polymorphism, Abstraction and Encapsulation.
- **Java is a robust language**
  - Java programs must be reliable because they are used in both consumer and mission-critical applications.
- **Java is a multithreaded language**
  - Java can perform many tasks at once by defining multiple threads.
  - It shares a common memory and other resources to execute multiple threads.
- **Portable**
  - Java Byte code can be carried to any platform. It doesn't require any implementation
- **Java programs can create applets**
  - Applets are programs of header files for creating a Java application.
  - Therefore, Java is a very successful language and it is gaining popularity day by day. that run in web browsers.
- **Java does not require any preprocessor**
  - It does not require inclusion of header files for creating a Java application.

Therefore, Java is a very successful language and it is gaining popularity day by day.

Difference

| Procedure-Oriented Programming | Object-Oriented Programming |
|---|---|
| Program is divided into small parts called **functions** | Program is divided into parts called **objects**. |
| It follows **Top Down approach**. | OOP follows **Bottom Up approach**. |
| Importance is not given to **data**. | Importance is given to the data rather than functions. |

**Object-Oriented Programming**.

- Procedural programming is about writing procedures or methods that perform operations on the data, while object-oriented programming is about creating objects that contain both data and methods.
- Object-oriented programming has several advantages over procedural programming:
- OOP is faster and easier to execute

- OOP provides a clear structure for the programs
- OOP helps to keep the Java code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time

**OOPs Concept:**

- Classes
- Objects
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

**Java - What are Classes and Objects?**

1. Classes and objects are the two main aspects of object-oriented programming.
2. Look at the following illustration to see the difference between class and objects:
3. Everything in Java is associated with classes and objects, along with its attributes and methods. For example: in real life, a car is an object. The car has attributes, such as weight and color, and methods, such as drive and brake.
4. You can access attributes by creating an object of the class, and by using the dot syntax (.):

**Encapsulation:**

Encapsulation in Java is a fundamental concept in object-oriented programming (OOP) that refers to the bundling of data and methods that operate on that data within a single unit, which is called a class in Java.

1. The meaning of **Encapsulation**, is to make sure that "sensitive" data is hidden from users. To achieve this, you must: declare class variables/attributes as private
2. provide public **get** and **set** methods to access and update the value of a private variable
3. The get method returns the value of the variable name.
4. The set method takes a parameter (newName) and assigns it to the name variable. The this keyword is used to refer to the current object.

**Java Inheritance (Subclass and Superclass):**

1. In Java, it is possible to inherit attributes and methods from one class to another. We group the "inheritance concept" into two categories:
2. **subclass** (child) - the class that inherits from another class
3. **superclass** (parent) - the class being inherited from
4. To inherit from a class, use the **extends** keyword.
5. In the example below, the Car class (subclass) inherits the attributes and methods from the Vehicle class (superclass):

**Polymorphism:**

1. Polymorphism refers to the ability to differentiate between entities with the same name.
2. Polymorphism in Java are mainly of 2 types:
    a. Overloading in Java
    b. Overriding in Java
3. For example, A single person is behaving differently at different time.

**Abstract Classes and Methods:**

1. Data **abstraction** is the process of hiding certain details and showing only essential information to the user.
   Abstraction can be achieved with either **abstract classes** or **interfaces**
2. The abstract keyword is a non-access modifier, used for classes and methods:
3. **Abstract class:** is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).
4. **Abstract method:** can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

**Lexical Issues in Java**

- Reserved Keywords and Identifiers.
- Case Sensitivity.
- Comments and Whitespace.
- String Escapes.
- Incorrect Operators and Punctuation.
- Improperly Closed Brackets and Parentheses.

**Identifiers:**

- *Identifiers* are names for variables, methods, classes, interfaces, or parameters.
- In Java, identifiers are used for identification purpose.
- Example: MyVariable, MYVARIABLE, _myvariable, name123

**Rules for naming an identifier**

1. All Identifiers must start with a letter ([**A-Z**],[**a-z**]), underscores '_' .
2. Java identifiers are **case-sensitive**.
3. White space in names is not permitted.
4. Identifiers should **not** start with digits(**[0-9]**).
5. No other Special symbols not allowed expect '_' Underscore

**Keywords:**

· Certain reserved words are called keywords.
· They can be briefly categorized into two parts : **keywords**(50) and **literals**(3).
· Keywords define functionalities and literals define a value.
· Example: break, int, float

**Case Sensitivity:**

The identifiers "myVariable" and "myvariable" are processed differently in Java because of the case-sensitivity of the language. It can be more difficult to read and maintain your code if the cases are mixed together inconsistently and can result in subtle problems.

**Comments:**

· There are two types to represent the comments, such as:
· Single line comments begin with the token // and end with a carriage return.
  For example, //this is the single-line syntax.
· Multi-Line comments begin with the token /* and end with the token */
  For example, /* this is multiline syntax*/

**String Escapes:**

- Special characters like newline ('\n') and double quotes ("") may be required when creating strings in Java. Syntax issues can occur when using these characters without the correct escapes.

**Improperly Closed Brackets and Parentheses**

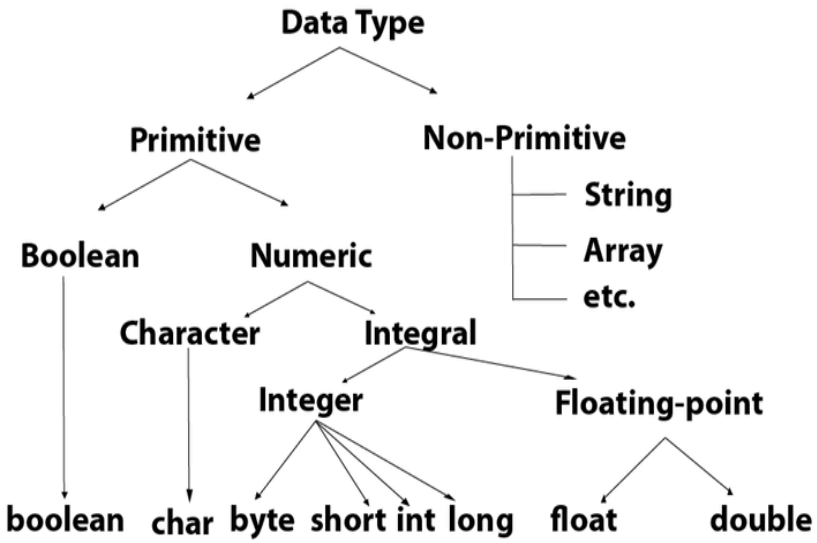
- Java syntax errors are frequently caused by mismatched or poorly closed brackets (" and ") and parentheses ('(' and ')').

**Incorrect Operators and Punctuation**

- Java utilizes a number of operators and punctuation marks, such as "+" for addition, "-" for subtraction, and ";" to end statements. A program may behave unexpectedly if these are misused or omitted, leading to compilation issues.

**Data Types in Java**

- Data types specify the different sizes and values that can be stored in the variable. There are two types of data types in Java:
- Primitive data types: The primitive data types include boolean, char, byte, short, int, long, float and double.
- Non-primitive data types: The non-primitive data types include Classes, Interfaces, and Arrays.




**Variables in Java**

- The value stored in a variable can be changed during program execution.
- Variables in Java are only a name given to a memory location. All the operations done on the variable affect that memory location.
- In Java, all variables must be declared before use.

**How to Declare Variables in Java?**


- **datatype:** Type of data that can be stored in this variable.
- **data_name:** Name was given to the variable.

**How to Initialize Variables in Java?**

- It can be perceived with the help of 3 components that are as follows:
- datatype: Type of data that can be stored in this variable.
- variable_name: Name given to the variable.
- value: It is the initial value stored in the variable.


**Arrays:**

- Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.
- Set of homogenous datatypes
- To declare an array, define the variable type with square brackets:
  Eg: String[] cars;
- We have now declared a variable that holds an array of strings. To insert values to it, you can place the values in a comma-separated list, inside curly braces:
  Eg: String[] cars={'Volvo','BMW','Ford','Mazda'};
- To create an array of integers, you could write:
  Eg: int[] num={10,20,30};

**Access the Elements of an Array:**

- You can access an array element by referring to the index number.
- This statement accesses the value of the first element in cars:

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
System.out.println(cars[0]);
// Outputs Volvo
```

Note: Array indexes start with 0: [0] is the first element. [1] is the second element, etc.

**Change an Array Element**

- To change the value of a specific element, refer to the index number:

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
cars[0] = "Opel";
System.out.println(cars[0]);
// Now outputs Opel instead of Volvo
```

**Array Length**

- **To find out how many elements an array has, use the length property:**

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
System.out.println(cars.length);
// Outputs 4
```

**Operators:**

- Operators in Java are the symbols used for performing specific operations in Java.
- There are multiple types of operators in Java all are mentioned below:
    - Arithmetic Operators
    - Unary Operators
    - Assignment Operator
    - Relational Operators
    - Logical Operators
    - Ternary Operator
    - Bitwise Operators
    - Shift Operators

**Arithmetic Operators**

| Operator | Name | Description | Example |
|---|---|---|---|
| + | Addition | Adds together two values | x + y |
| - | Subtraction | Subtracts one value from another | x - y |
| * | Multiplication | Multiplies two values | x * y |
| / | Division | Divides one value by another | x / y |
| % | Modulus | Returns the division remainder | x % y |
| ++ | Increment | Increases the value of a variable by 1 | ++x |
| -- | Decrement | Decreases the value of a variable by 1 | --x |

**Java Assignment Operators**

| Operator | Example | Same As |
|---|---|---|
| = | x = 5 | x = 5 |
| += | x += 3 | x = x + 3 |
| -= | x -= 3 | x = x - 3 |
| *= | x *= 3 | x = x * 3 |
| /= | x /= 3 | x = x / 3 |
| %= | x %= 3 | x = x % 3 |
| &= | x &= 3 | x = x & 3 |
| \|= | x \|= 3 | x = x \| 3 |
| ^= | x ^= 3 | x = x ^ 3 |
| >>= | x >>= 3 | x = x >> 3 |
| <<= | x <<= 3 | x = x << 3 |

- **Java Relational Operators** are a bunch of binary operators used to check for relations between two operands, including equality, greater than, less than, etc.
- **Java Logical Operators**
    - You can also test for true or false values with logical operators.
    - Logical operators are used to determine the logic between variables or values:

| Operator | Name | Description | Example |
|---|---|---|---|
| && | Logical and | Returns true if both statements are true | x < 5 && x < 10 |
| \|\| | Logical or | Returns true if one of the statements is true | x < 5 \|\| x < 4 |
| ! | Logical not | Reverse the result, returns false if the result is true | !(x < 5 && x < 10) |

**Unary Operator:**

- Unary operators need only one operand. They are used to increment, decrement, or negate a value.
- : Unary minus, used for negating the values.
- + : Unary plus indicates the positive value (numbers are positive without this, however). It performs an automatic conversion to int when the type of its operand is the byte, char, or short. This is called unary numeric promotion.
- ++ : Increment operator, used for incrementing the value by 1. There are two varieties of increment operators.
  – Post-Increment: Value is first used for computing the result and then incremented.
  – Pre-Increment: Value is incremented first, and then the result is computed.
  -- : Decrement operator, used for decrementing the value by 1. There are two varieties of decrement operators.
  – Post-decrement: Value is first used for computing the result and then decremented.
  – Pre-Decrement: The value is decremented first, and then the result is computed.
- ! : Logical not operator, used for inverting a boolean value.

**Ternary Operator in Java**

- variable = Expression1 ? Expression2: Expression3
- if(Expression1)
  {
  variable = Expression2;
  }
  else
  {
  variable = Expression3;
  }

```java
import java.io.*;

class Ternary {
    public static void main(String[] args)
    {

        // variable declaration
        int n1 = 5, n2 = 10, max;

        System.out.println("First num: " + n1);
        System.out.println("Second num: " + n2);

        // Largest among n1 and n2
        max = (n1 > n2) ? n1 : n2;

        // Print the largest number
        System.out.println("Maximum is = " + max);
    }
}
```

- **Output**

First num: 5

Second num: 10

Maximum is = 10

**Bitwise Operators:**

- Bitwise operators are used to performing the manipulation of individual bits of a number.
- Bitwise OR (|)
- This operator is a binary operator, denoted by '|'. It returns bit by bit OR of input values, i.e., if either of the bits is 1, it gives 1, else it shows 0.

```
a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)

Bitwise OR Operation of 5 and 7
  0101
| 0111
 ________
  0111   = 7 (In decimal)
```

**Shift Operator:**

| Name of operator | Sign | Description |
|---|---|---|
| Signed Left Shift | << | The left shift operator moves all bits by a given number of bits to the left. |
| Signed Right Shift | >> | The right shift operator moves all bits by a given number of bits to the right. |

**Creating Program:**

We can create a program using Text Editor (Notepad) or IDE (NetBeans)

```
class Test
{
        public static void main(String []args)
        {
            System.out.println("My First Java Program.");
        }
};
```

File -> Save -> d:\Test.java

• Compiling the program

1. To compile the program, we must run the Java compiler (javac), with the name of the source file on "command prompt" like as follows

2. If everything is OK, the "javac" compiler creates a file called "Test.class" containing byte code of the program.

**d:>javac Test.java**

• Running the program

We need to use the Java Interpreter to run a program.

**d:>java Test**

**class :** class keyword is used to declare classes in Java
**public :** It is an access specifier. Public means this function is visible to all.
**static :** static is again a keyword used to make a function static. To execute a static function you do not have to create an Object of the class. The main() method here is called by JVM, without creating any object for class.
**void :** It is the return type, meaning this function will not return anything.

**main :** main() method is the most important method in a Java program. This is the method which is executed, hence all the logic must be inside the main() method. If a java class is not having a main() method, it causes compilation error.
**String[] args :** This is used to signify that the user may opt to enter parameters to the Java Program at command line. We can use both String[] args or String args[]. Java compiler would accept both forms.
**System.out.println :** This is used to print anything on the console like "printf" in C language.

**Control Statements:**

Java compiler executes the code from top to bottom. The statements in the code are executed according to the order in which they appear. However, Java provides statements that can be used to control the flow of Java code. Such statements are called control flow statements.

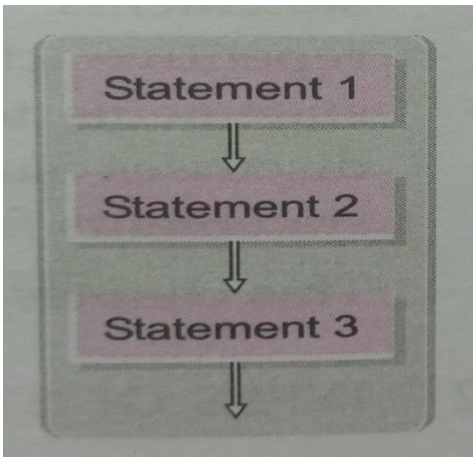
Control Statements can be classified into three types.

1. Sequence
2. Selection
3. Iteration

**Sequence:**

Begins from the first statement.

All Statements should be executed one after another(Normal Flow of Control)




**Syntax:**

Statement1

[Statement2]

[Statement3]

Example Program:

```
class Test
{
   public static void main(String []args)
   {
      System.out.println("Hello World");
   }
};
```

**Selection:**

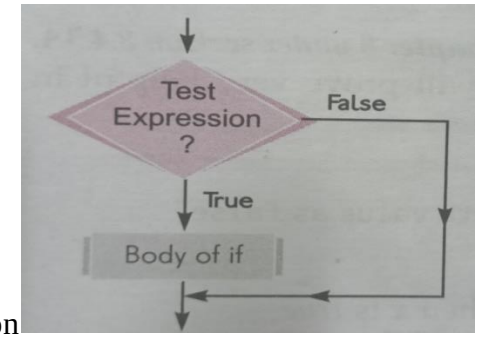
Statements should be executed depending upon a condition-test, also called decision construct[if the condition becomes True→ one set of statements executed, False→another set of statements executed)

Real Time Example:Traffic Signal

**If Condition:**

- If condition return True → a course of statement should be executed.

  False→ a course of statements should be ignored.




- Operators will help when checking the condition

Syntax:

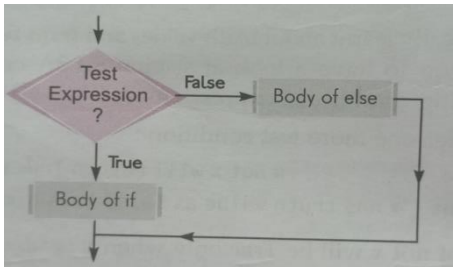
```
if(condition)
{
  // Statements to execute if
  // condition is true
}
```

**Example :**

```
Class ifex
{
public static void main(String[] args)
{
int a=15;
if(a>=10 && a<=99)
{
System.out.println("Double Digit")
}
}
```

**If …Else:**

The block below if gets executed if the condition evaluates to True, and the block below else get executed if the condition evaluates to False.




```
if (condition)
{
   // Executes this block if
   // condition is true
}
else
{
   // Executes this block if
   // condition is false
}
```

**Example :**

```
Class ifelseex
```

```
{
public static void main(String[] args)
{
int a=15;
int b=20
if(a>b)
{
System.out.println(“a is greater”)
}
else
{
System.out.println(“b is greater”)
}
}
```

**If..elseif Ladder:**

**Syntax:**

```
if (condition)
   statement 1;
else if (condition)
   statement 2;
.
.
else
   statement;
```

**Example :**

```
Class ifelseifex
{
public static void main(String[] args)
{
int a=15;
int b=20;
int c=30;
if((a>b)&&(a>c))
```

{

System.out.println(“a is greater”)

}

elseif((b>a)&&(b>c))

{

System.out.println(“b is greater”)

}

else

{

System.out.println(“c is greater”)

}

}

**Repetition Iteration(Looping):**

Repetition of a set of statements depending upon a condition. The block statement should be executed again and again until the condition becomes false.

Real –Time Example: Dosa Making

Looping in programming languages is a feature which facilitates the execution of a set of instructions/functions repeatedly while some condition evaluates to true. Java provides three ways for executing the loops. While all the ways provide similar basic functionality, they differ in their syntax and condition checking time.

**While Loop:**

A while loop is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition.

**Syntax :**

```
while (boolean condition)
{
  loop statements...
}
```

Example:

```
import java.io.*;

class whilel {
   public static void main (String[] args) {
     int i=1;
     while (i<=5)
     {
       System.out.println(i);
       i++;
```

```
    }
  }
}
```

Output :

1

**2**

**3**

**4**

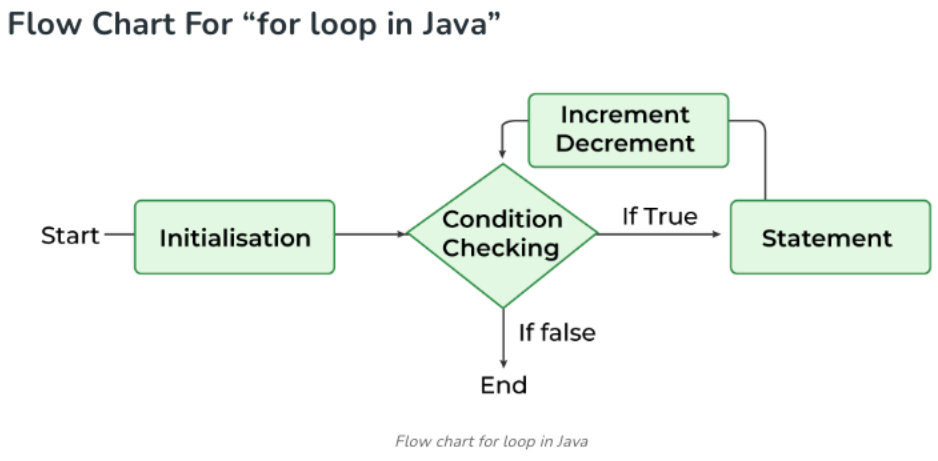
**5**

**For Loop:**

**Loops in Java** come into use when we need to repeatedly execute a block of statements. **Java for loop** provides a concise way of writing the loop structure. The for statement consumes the initialization, condition, and increment/decrement in one line thereby providing a shorter, easy-to-debug structure of looping.

**Parts of Java For Loop**
Java for loop is divided into various parts as mentioned below:

- Initialization Expression
- Test Expression
- Update Expression
  - **Syntax:**
  - for (initialization expr; test expr; update exp)
    {
        // body of the loop
        // statements we want to execute
    }




*Flow chart for loop in Java*

```
Class forl {

  public static void main(String[] args)
  {
    for (int i = 1; i <= 5; i++) {
      System.out.println(i);
    }
  }
```

```
}
```

Output :

1

**2**

**3**

**4**

**5**

**do…While** :

Java **do-while loop** is an **Exit control loop**. Therefore, unlike ***for*** or ***while*** loop, a do-while check for the condition after executing the statements of the loop body.

**Syntax:**

```
do
{
   // Loop Body
   Update_expression
}
// Condition check
while (test_expression);
```

Example:

```
import java.io.*;

class whilel {
   public static void main (String[] args) {
     int i=1;

     {
       System.out.println(i);
       i++;
     } while (i<=5);
   }
}
```

Output :

1
2
3
4
5

**Break Statement:**

Break Statement is a loop control statement that is used to terminate the loop. As soon as the break statement is encountered from within a loop, the loop iterations stop there, and control returns from the loop immediately to the first statement after the loop

```
Class forbr {

   public static void main(String[] args)
   {
      for (int i = 1; i <= 5; i++) {
        if(i==3){
          break;
          }
          System.out.println(i);
      }
   }
}
```

Output:

1

2

**Continue Statement:**

when a continue statement is encountered the control directly jumps to the beginning of the loop for the next iteration instead of executing the statements of the current iteration. The continue statement is used when we want to skip a particular condition and continue the rest execution.

```
Class forcon{

   public static void main(String[] args)
   {
      for (int i = 1; i <= 5; i++) {
        if(i==3){
          continue;
          }
          System.out.println(i);
      }
   }
}
```

Output:

1

2

4

5